

Практическая работа №2

Поддержка и тестирование программных модулей
на тему: Создание приложения калькулятор

Специальность «Информационные системы и программирование»

Руководитель

_____ / Д.Б. Берестов/

Оценка _____

«_19_» _Март_ 2026 г.

Исполнитель

_____ / К.В. Кочетков/

Группа ИПо-22

«_19_» _Март_ 2026 г.

Пермь, 2026

Оглавление

Введение	2
В данной практической работе я использовал:	2
1. Среду разработки:	2
2. Технологии и фреймворки:	2
3. Язык программирования:	2
4. Архитектура приложения:.....	2
5. Ключевые особенности кода:	3
6. Тестирование:	3
Ход работы:	3
1. Создание пустого решения	3
2. Создание Библиотеки классов	4
3. Логика калькулятора.....	5
4. Создание WPF	6
5. Создание интерфейса калькулятора.....	7
6. Написание логики интерфейса	8
7. Создание проекта для модульных тестов	10
8. Запуск тестов	14
Заключение:	15

Введение

Visual Studio интегрированная среда разработки для создания многофункциональных, привлекательных кроссплатформенных приложений для Windows, Mac, Linux, iOS и Android.

В данной практической работе я использовал:

1. Среда разработки:

Microsoft Visual Studio 2022 – интегрированная среда разработки (IDE) для создания приложений на языке C#

2. Технологии и фреймворки:

WPF (Windows Presentation Foundation) – технология для создания графического интерфейса пользователя. Использована для разработки окна калькулятора, кнопок и полей ввода.

.NET Framework – программная платформа, на которой работает приложение.

3. Язык программирования:

C# – объектно-ориентированный язык программирования. На нем написана вся логика приложения.

4. Архитектура приложения:

Библиотека классов (Class Library) отдельный проект CalculatorLibrary, содержащий основную логику вычислений. Это позволяет отделить бизнес-логику от интерфейса.

WPF приложение проект CalculatorWPF, отвечающий за отображение интерфейса и взаимодействие с пользователем.

Модульное тестирование (Unit Testing) проект CalculatorTests, содержащий тесты для проверки корректности работы методов калькулятора.

5. Ключевые особенности кода:

Использование **динамического типа** `dynamic` в методе `Execute` для поддержки различных числовых типов данных (`int`, `double`).

Реализация **статического метода** для вычислений, что позволяет вызывать его без создания экземпляра класса.

Поддержка арифметических операций: `*`, `+`, `-`, `,`, `/`, `^` (возведение в степень).

Обработка деления на ноль с возвратом значения **бесконечность** (∞).

6. Тестирование:

Фреймворк **MSTest** для написания и запуска модульных тестов.

Написаны тесты для проверки всех арифметических операций, включая деление на ноль.

Ход работы:

1. Создание пустого решения

Первым шагом я запустил Microsoft Visual Studio 2022. В главном меню выбрал **Файл** → **Создать** → **Проект**. В поисковой строке ввел "Пустой проект" и выбрал шаблон **"Новое решение"**, который позволяет создать пустой контейнер для дальнейшего добавления проектов. Указал имя решения **KochetkovCalc**, выбрал папку для сохранения и нажал кнопку "Создать". В результате было создано пустое решение, которое стало основой для всех последующих проектов.

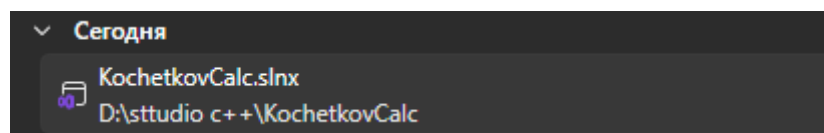


Рисунок 1 «Создание пустого проекта»

2. Создание Библиотеки классов

Далее я создал проект библиотеки классов, который будет содержать основную логику вычислений.

Для этого нажал правой кнопкой мыши на решение, выбрал «Добавить» «Новый проект», в поиске нашел шаблон «Библиотека классов (.NET Framework)» и выбрал его.

Указал имя проекта «**CalculatorLibrary**» и нажал «Создать». Этот проект будет отвечать за выполнение арифметических операций.

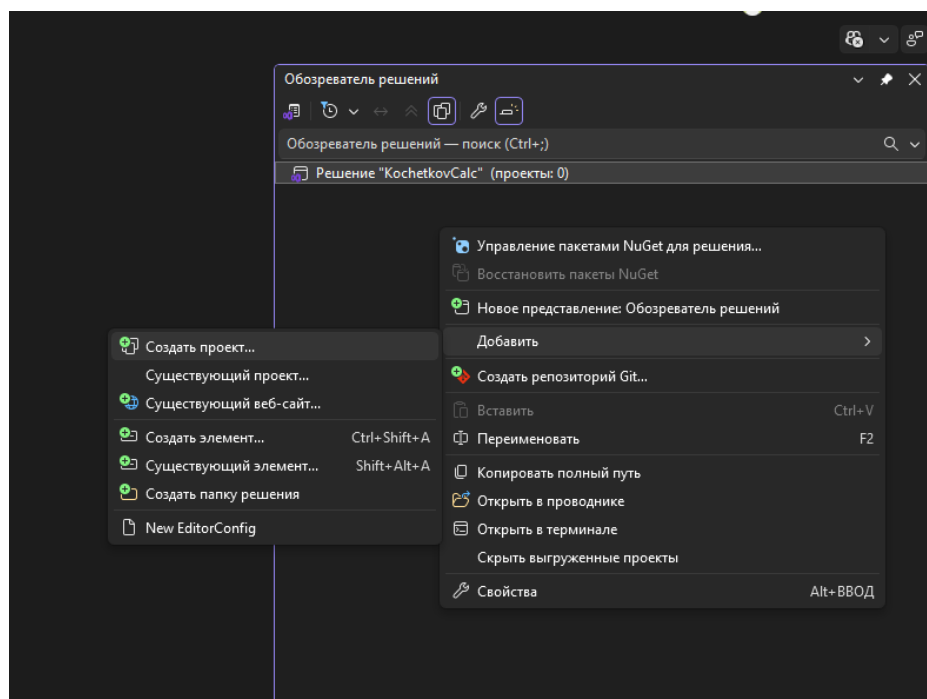


Рисунок 2 «Добавление проэкта библиотеки»

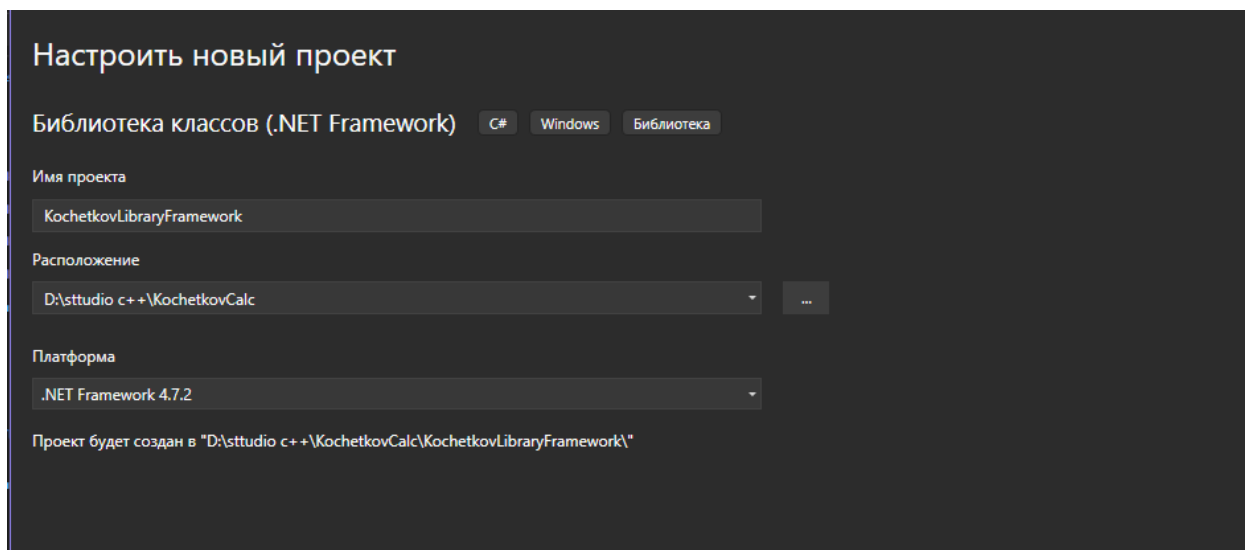


Рисунок 3 «Добавление библиотеки»

После создания проекта я удалил автоматически сгенерированный файл «**Class1.cs**», так как он не требуется для дальнейшей работы.

Затем я добавил новый класс, который будет содержать методы для вычислений. Для этого нажал правой кнопкой на проект «**KochetkovLibraryFramework**», выбрал «Добавить» «Класс» и назвал его «**KochetkovClass.cs**».

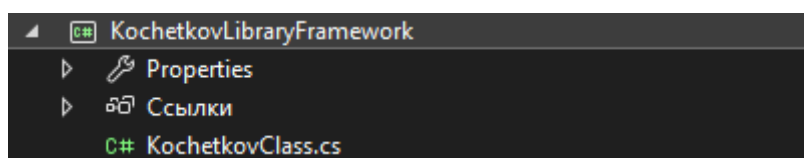


Рисунок 4 «Добавление класса»

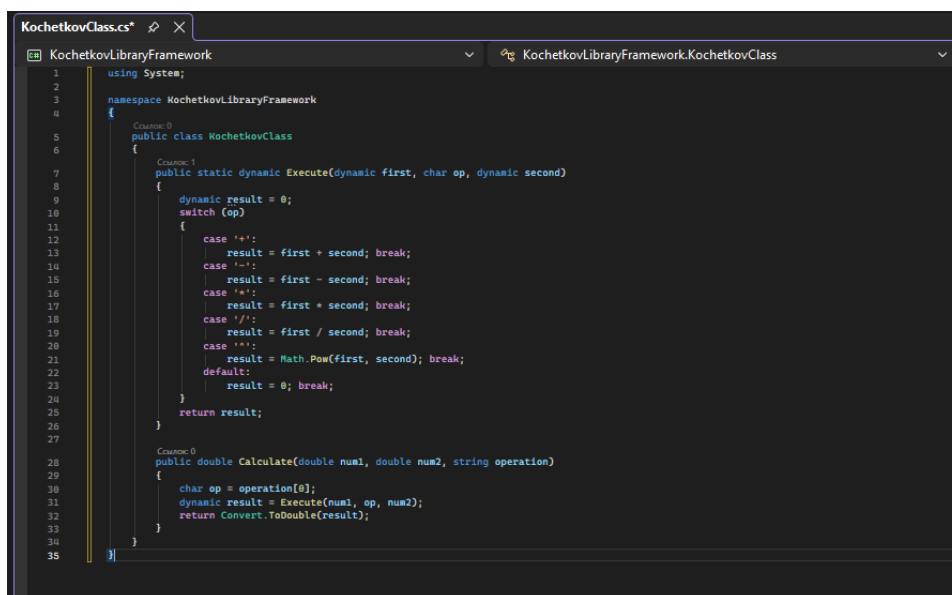
3. Логика калькулятора

Далее В файл **Kochetkovclass.cs** я поместил код калькулятора.

Главный метод **Execute** реализован как статический с использованием типа **dynamic**, что позволяет принимать числа разных типов (**int**, **double**).

Метод принимает два числа и символ операции, после чего с помощью конструкции **switch** выполняет соответствующее действие: сложение, вычитание, умножение, деление или возведение в степень.

Для удобства вызова из **WPF** приложения я также добавил метод **Calculate**, который преобразует строковый оператор в символ и вызывает метод **Execute**.



```
1 using System;
2
3 namespace KochetkovLibraryFramework
4 {
5     Создать: 0
6     public class KochetkovClass
7     {
8         Создать: 1
9         public static dynamic Execute(dynamic first, char op, dynamic second)
10         {
11             dynamic result = 0;
12             switch (op)
13             {
14                 case '+':
15                     result = first + second; break;
16                 case '-':
17                     result = first - second; break;
18                 case '*':
19                     result = first * second; break;
20                 case '/':
21                     result = first / second; break;
22                 case '^':
23                     result = Math.Pow(first, second); break;
24                 default:
25                     result = 0; break;
26             }
27             return result;
28         }
29
30         Создать: 0
31         public double Calculate(double num1, double num2, string operation)
32         {
33             char op = operation[0];
34             dynamic result = Execute(num1, op, num2);
35             return Convert.ToDouble(result);
36         }
37     }
38 }
```

Рисунок 5 «Код»

4. Создание WPF

Следующим этапом я создал проект для графического интерфейса.

Снова нажал правой кнопкой на решение, выбрал «Добавить» «Новый проект», в поиске ввел "WPF" и выбрал шаблон «**Приложение WPF (.NET Framework)**». Указал имя проекта «**KochetkovWPF**» и нажал «Создать». Этот шаблон позволяет создавать оконные приложения с богатым графическим интерфейсом.

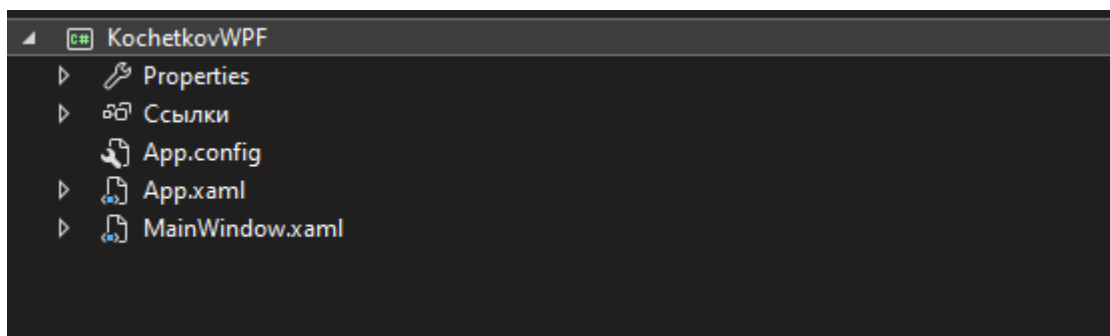


Рисунок 6 «Создание WPF проекта»

После создания **WPF проекта** необходимо было подключить ранее созданную библиотеку, чтобы приложение могло использовать ее методы.

Для этого я нажал правой кнопкой на проект «**KochetkovWPF**», выбрал «Добавить» «Ссылка», в открывшемся окне перешел на вкладку «**Проекты**», поставил галочку напротив «**KochetkovLibraryFramework**» и нажал «ОК». После этого библиотека стала доступна для использования в WPF приложении.

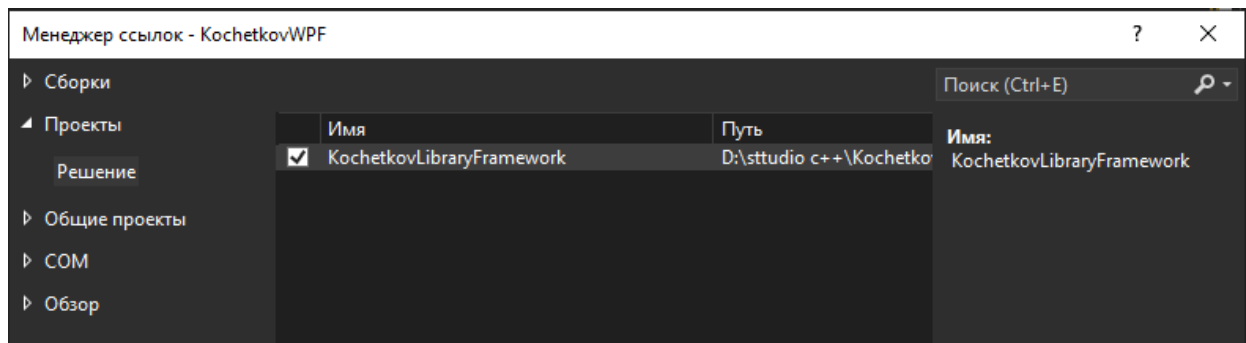


Рисунок 7 «Добавление библиотеки»

5. Создание интерфейса калькулятора

Для создания интерфейса я открыл файл MainWindow.xaml.

Поле ввода (TextBox) – расположено в верхней части окна, служит для отображения вводимых чисел и результата вычислений;

Поле результата (TextBox) – расположено под полем ввода, отображает пример вычисления (например, "3 + 3 =");

Сетка кнопок – создана с помощью элемента Grid с разбиением на 5 строк и 4 столбца. В ячейки сетки размещены кнопки: цифры от 0 до 9, кнопки операций (+, -, *, /, ^), кнопка точки, кнопка очистки C и кнопка равно =.

Каждой кнопке был назначен обработчик события Click.

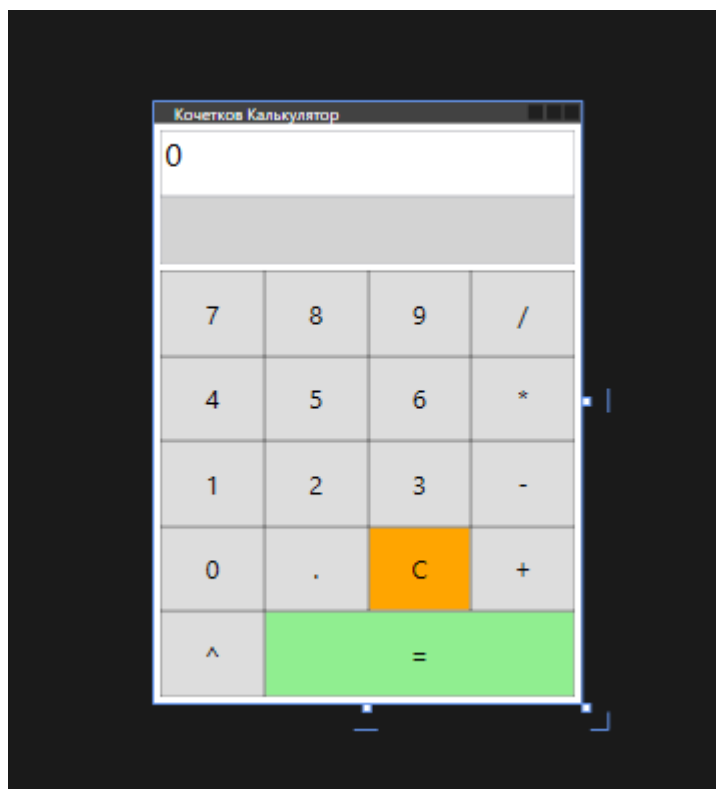


Рисунок 8 «Интерфейс калькулятора»

6. Написание логики интерфейса

В файле «MainWindow.xaml.cs» я написал логику работы калькулятора. Для хранения состояния объявил переменные:

- «**number1**» первое число;
- «**operation**» выбранная операция;
- «**newNumber**» флаг нового числа.

Затем реализовал обработчики:

- «**Button_Click**» добавляет цифры в поле ввода;
- «**Dot_Click**» добавляет десятичную точку;
- «**Operator_Click**» сохраняет первое число и операцию;
- «**Equals_Click**» вычисляет результат через библиотеку;
- «**Clear_Click**» очищает все поля.

Для корректной работы с десятичными числами использовал «**CultureInfo.InvariantCulture**», что позволяет вводить числа через точку.

```

1 using System;
2 using System.Windows;
3 using System.Windows.Controls;
4 using KochetkovLibraryFramework; //моя библиотека
5
6 namespace KochetkovWPF
7 {
8     Схема 2
9     public partial class MainWindow : Window
10     {
11         //хранение первого введенного числа
12         double number1 = 0;
13         //хранение операции
14         string operation = "";
15         // true значит новое число false продолж ввод текущ
16         bool newNumber = true;
17         KochetkovClass calculator = new KochetkovClass();
18         //конструктор окна
19         public MainWindow()
20         {
21             InitializeComponent();
22         }
23         Схема 10
24         //обработчик цифр
25         private void Button_Click(object sender, RoutedEventArgs e)
26         {
27             Button button = (Button)sender;
28
29             //новое число - очищаем поле
30             if (newNumber)
31             {
32                 InputTextBox.Text = ""; //очистка поля
33                 newNumber = false; //ввод числа
34             }
35             //добавляем нажатую цифру в конец
36             InputTextBox.Text = InputTextBox.Text + button.Content.ToString();
37
38             Схема 1
39             //обработчик точки для десятичных дробей
40             private void Dot_Click(object sender, RoutedEventArgs e)
41             {
42                 //если новое число - ставим 0 перед точкой
43                 if (newNumber)
44                 {
45                     InputTextBox.Text = "0"; //остав 0
46                     newNumber = false; //продолж ввод
47                 }
48                 //добавляем точку если её нет в числе
49                 if (!InputTextBox.Text.Contains("."))
50                 {
51                     InputTextBox.Text = InputTextBox.Text + ".";
52                 }
53             }
54             Схема 3
55             //операций
56             private void Operator_Click(object sender, RoutedEventArgs e)
57             {
58                 Button button = (Button)sender;
59                 //есть введенное число
60                 if (!newNumber)
61                 {
62                     //сохраняем текущее число как первое
63                     number1 = Convert.ToDouble(InputTextBox.Text, System.Globalization.CultureInfo.InvariantCulture);
64                     newNumber = true; //след число будет новым
65                 }
66                 //запоминаем выбранную операцию
67                 operation = button.Content.ToString();
68                 //отображ пример в поле sonucu
69                 ResultTextBox.Text = number1.ToString() + " * " + operation;
70             }
71             Схема 1
72             //обработчик равно
73             private void Equals_Click(object sender, RoutedEventArgs e)
74             {
75
76             }
77         }
78     }
79 }

```

Рисунок 9 «Часть код MainWindow.xaml.cs»

```

1 <ColumnDefinition Width="*" />
2 </Grid.ColumnDefinitions>
3
4 <!-- 1ряд -->
5 <Button Content="7" Grid.Row="0" Grid.Column="0" Click="Button_Click" FontSize="20" />
6 <Button Content="8" Grid.Row="0" Grid.Column="1" Click="Button_Click" FontSize="20" />
7 <Button Content="9" Grid.Row="0" Grid.Column="2" Click="Button_Click" FontSize="20" />
8 <Button Content="/" Grid.Row="0" Grid.Column="3" Click="Operator_Click" FontSize="20" />
9
10 <!-- 2ряд -->
11 <Button Content="4" Grid.Row="1" Grid.Column="0" Click="Button_Click" FontSize="20" />
12 <Button Content="5" Grid.Row="1" Grid.Column="1" Click="Button_Click" FontSize="20" />
13 <Button Content="6" Grid.Row="1" Grid.Column="2" Click="Button_Click" FontSize="20" />
14 <Button Content="*" Grid.Row="1" Grid.Column="3" Click="Operator_Click" FontSize="20" />
15
16 <!-- 3ряд -->
17 <Button Content="1" Grid.Row="2" Grid.Column="0" Click="Button_Click" FontSize="20" />
18 <Button Content="2" Grid.Row="2" Grid.Column="1" Click="Button_Click" FontSize="20" />
19 <Button Content="3" Grid.Row="2" Grid.Column="2" Click="Button_Click" FontSize="20" />
20 <Button Content="-" Grid.Row="2" Grid.Column="3" Click="Operator_Click" FontSize="20" />
21
22 <!-- 4ряд -->
23 <Button Content="0" Grid.Row="3" Grid.Column="0" Click="Button_Click" FontSize="20" />
24 <Button Content="." Grid.Row="3" Grid.Column="1" Click="Dot_Click" FontSize="20" />
25 <Button Content="C" Grid.Row="3" Grid.Column="2" Click="Clear_Click" FontSize="20" Background="Orange" />
26 <Button Content="+" Grid.Row="3" Grid.Column="3" Click="Operator_Click" FontSize="20" />
27
28 <!-- 5ряд -->
29 <Button Content="=" Grid.Row="4" Grid.Column="0" Click="Operator_Click" FontSize="20" />
30 <Button Content="=" Grid.Row="4" Grid.Column="1" Grid.ColumnSpan="3"
31     Click="Equals_Click" FontSize="20" Background="LightGreen" />
32
33 </Grid>
34 </Grid>
35 </Window>

```

Рисунок 10 «Часть кода MainWindow.xaml»

Далее я назначил проект «**KochetkovWPF**» запускаемым, нажал «**Ctrl + Shift + B**» для сборки решения и затем «**F5**» для запуска. Программа успешно запустилась.

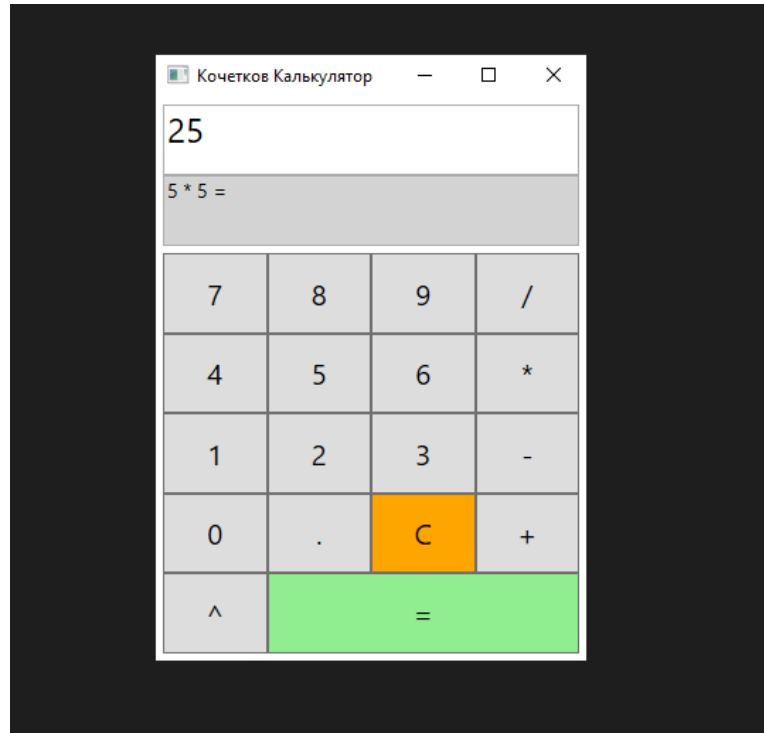


Рисунок 11 «Запуск калькулятора»

Я проверил работу калькулятора.

Все операции выполняются корректно.

Десятичные числа вводятся через точку.

При делении на ноль выводится символ бесконечности «...».

7. Создание проекта для модульных тестов

Для проверки корректности работы методов калькулятора я создал проект модульных тестов. Для этого нажал правой кнопкой на решение, выбрал «Добавить» «Новый проект», в поиске ввел «**Проект модульного тестирования**» и выбрал шаблон «**Проект модульного теста (.NET Framework)**». Указал имя проекта «**KochetkovTests**» и нажал «Создать».

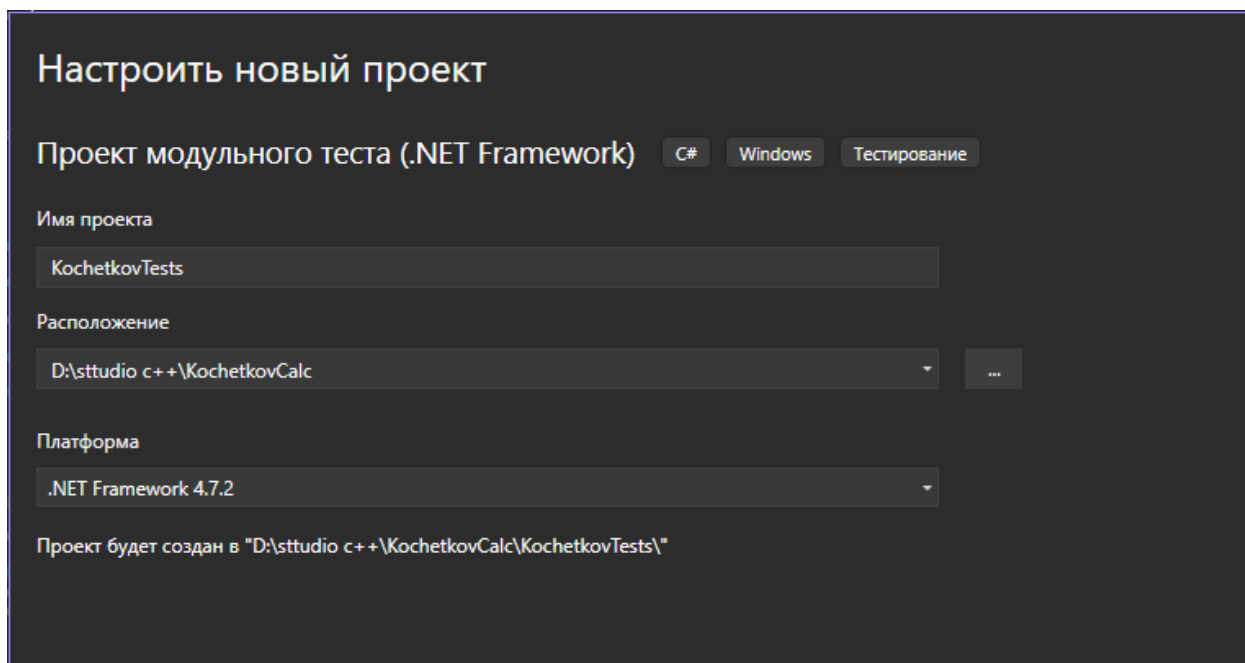


Рисунок 12 «Создание проекта модульного тестирования»

Чтобы тесты могли использовать методы библиотеки, я добавил ссылку на проект «**KochetkovLibraryFramework**». Для этого нажал правой кнопкой на «**KochetkovTests**», выбрал «Добавить» «Ссылка», перешел на вкладку «Проекты», поставил галочку напротив «**KochetkovLibraryFramework**» и нажал «ОК».

Для работы с типом «**dynamic**», который используется в коде тестов, потребовалось добавить ссылку на сборку «**Microsoft.Csharp**». Без этой ссылки компилятор не может обработать ключевое слово «**dynamic**».

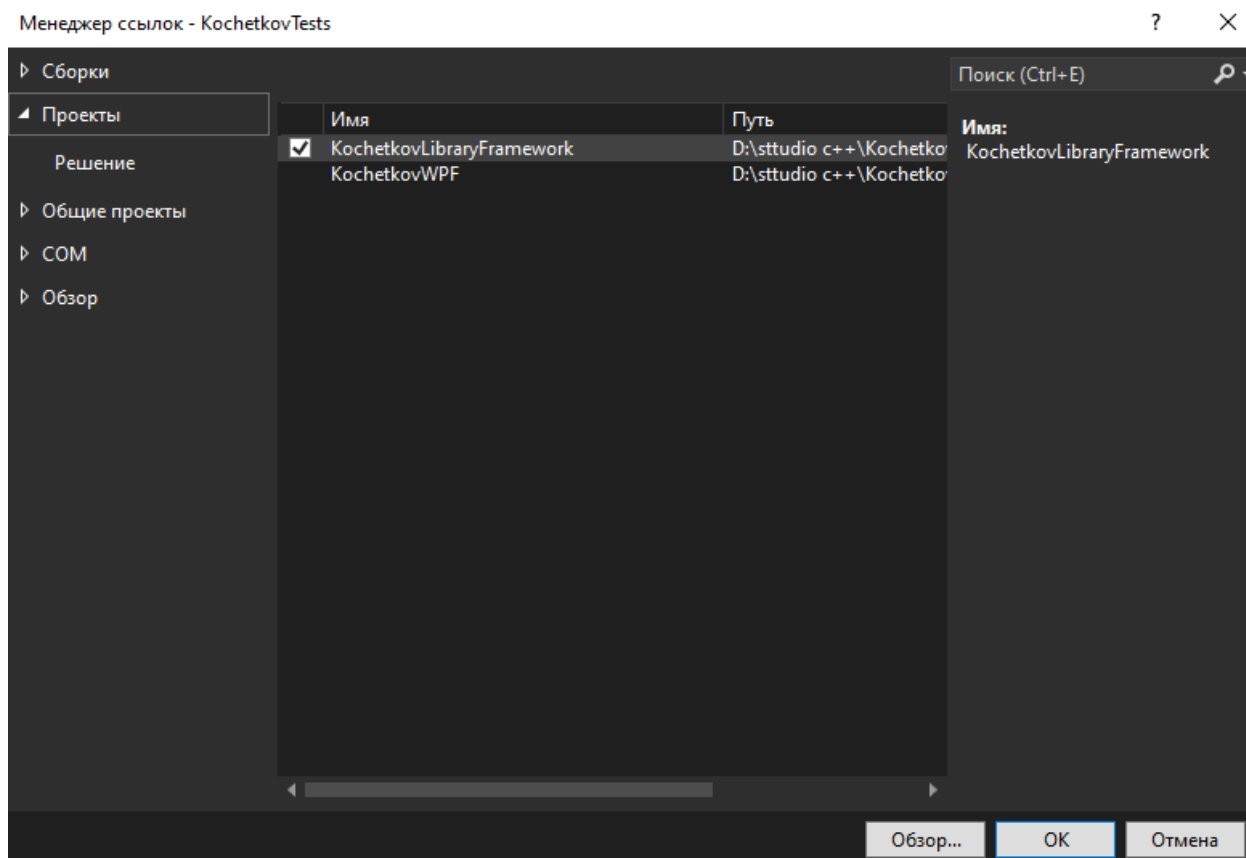


Рисунок 13 «Добавление ссылки на библиотеку»

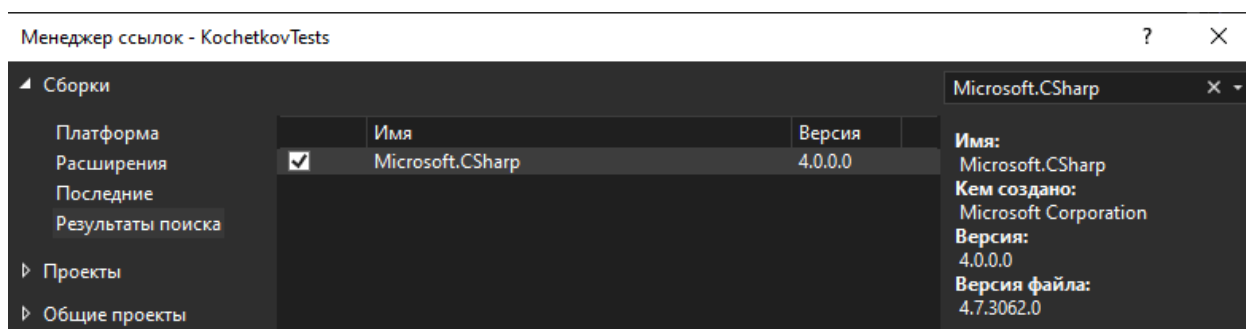


Рисунок 14 «Добавление ссылки Microsoft.Csharp»

Я удалил автоматически созданный файл «**UnitTest1.cs**», так как он не требуется для дальнейшей работы.

Затем я создал новый класс для тестов. Нажал правой кнопкой на проект «**KochetkovTests**», выбрал «Добавить» «Класс» и назвал его «**KochetkovClassTests.cs**».

В файл «**KochetkovClassTests.cs**» я написал тесты для проверки всех операций:

«**test_3plus3**» проверяет сложение ($3 + 3 = 6$);

«test_10minus4» проверяет вычитание ($10 - 4 = 6$);

«test_6times7» проверяет умножение ($6 \times 7 = 42$);

«test_15divide3» проверяет деление ($15 \div 3 = 5$);

«test_2to3» проверяет возведение в степень ($2^3 = 8$);

«test_10divide0» проверяет деление на ноль ($10 \div 0 = \dots$);

«test_calculate_5plus3» проверяет метод calculate ($5 + 3 = 8$).

Каждый тест помечен атрибутом «**[TestMethod]**», а сам класс атрибутом «**[TestClass]**». Для проверки результатов используется метод «**Assert.AreEqual**».

```
//сложение
[TestMethod]
// Ссылка 0
public void test_3plus3()
{
    dynamic result = KorchetkovClass.Execute(3, '+', 3);
    Assert.AreEqual(6, result);
}

//вычитание
[TestMethod]
// Ссылка 0
public void test_10minus4()
{
    dynamic result = KorchetkovClass.Execute(10, '-', 4);
    Assert.AreEqual(6, result);
}

//умножение
[TestMethod]
// Ссылка 0
public void test_6times7()
{
    dynamic result = KorchetkovClass.Execute(6, '*', 7);
    Assert.AreEqual(42, result);
}

//деление
[TestMethod]
// Ссылка 0
public void test_15divide3()
{
    dynamic result = KorchetkovClass.Execute(15, '/', 3);
    Assert.AreEqual(5, result);
}

//степень
[TestMethod]
// Ссылка 0
public void test_2to3()
{
    dynamic result = KorchetkovClass.Execute(2, '^', 3);
    Assert.AreEqual(8, result);
}

//деление на ноль
[TestMethod]
// Ссылка 0
public void test_10divide0()
{
    dynamic result = KorchetkovClass.Execute(10, '/', 0);
    Assert.AreEqual(double.PositiveInfinity, result);
}

//метод calculate
[TestMethod]
// Ссылка 0
public void test_calculate_5plus3()
{
    KorchetkovClass calculator = new KorchetkovClass();
    double result = calculator.Calculate(5, 3, "+");
    Assert.AreEqual(8, result);
}
```

Рисунок 14 «Код тестов»

8. Запуск тестов

После написания тестов в файле «**KochetkovClassTests.cs**» я запустил их выполнение.

Для этого открыл окно Обозреватель Тестов («Тест» - «Обозреватель тестов») и нажал кнопку "Запустить все тесты".

Все 7 тестов успешно прошли (зеленые галочки).

Результаты тестирования:

- test_5plus3 – сложение $3 + 3 = 6$
- test_10minus4 – вычитание $10 - 4 = 6$
- test_6times7 – умножение $6 \times 7 = 42$
- test_15divide3 – деление $15 \div 3 = 5$
- test_2to3 – степень $2^3 = 8$
- test_10divide0 – деление на ноль $10 \div 0 = \dots$
- test_calculate_5plus3 – метод Calculate $5 + 3 = 8$

Все тесты пройдены успешно, что подтверждает корректную работу библиотеки «**KochetkovLibraryFramework**».

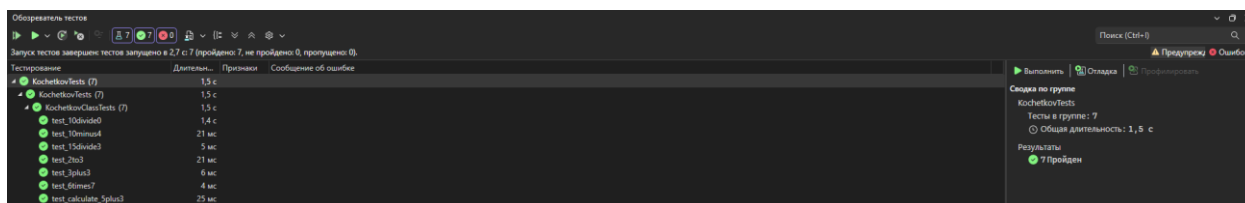


Рисунок 15 «Тестирование»

Заключение:

В ходе выполнения практической работы я разработал приложение "Калькулятор" с графическим интерфейсом на технологии «WPF». В процессе работы я узнал много нового и полезного.

Я научился:

- 1)Создавать проекты в Visual Studio (библиотеку классов, WPF приложение, проект тестов);
- 2)Разделять логику программы и интерфейс, вынося вычисления в отдельную библиотеку классов;
- 3)Работать с технологией «WPF» для создания графического интерфейса;
- 4)Использовать тип «dynamic» для гибкой работы с разными типами чисел;
- 5)Обрабатывать десятичные числа с помощью «CultureInfo.InvariantCulture»;
- 6)Реализовывать обработку деления на ноль с выводом трех точек;
- 7)Писать модульные тесты с помощью «MSTest» для проверки работы методов;
- 8)Запускать тесты через «Обозреватель тестов» и анализировать результаты.

Мне понравилось работать над этим проектом, так как я смог применить полученные знания на практике и создать полностью работающее приложение. Особенно интересно было настраивать взаимодействие между библиотекой и «WPF» приложением, а также писать тесты, которые автоматически проверяют корректность работы всех операций.

В результате все задачи выполнены, программа работает правильно, а тесты успешно проходят. Этот опыт поможет мне в дальнейшей разработке более сложных приложений.

